

REPORTE DE PRÁCTICA: ARREGLOS BIDIMENSIONALES Y LÓGICA DE ESTADOS

Asignatura: Programación Estructurada
Estudiante: Aldo Ezequiel Rodriguez Godoy
Código: 224097617
Sección: D04
Profesor: JORGE ERNESTO LOPEZ ARCE DELGADO
Fecha: 18 de marzo del 2026

INTRODUCCIÓN

En el desarrollo de software estructurado, cuando los datos superan la dimensionalidad lineal de los vectores tradicionales, se recurre al uso de arreglos bidimensionales, comúnmente conocidos como matrices o tablas. Una matriz en el lenguaje C representa una estructura de datos estática, homogénea y organizada de forma tabular mediante dos componentes de localización: un índice para las filas y un índice para las columnas.

Aunque conceptualmente visualizamos una matriz como una cuadrícula de dos dimensiones, en la memoria física RAM el almacenamiento se mapea de forma completamente lineal y continua siguiendo el orden de fila mayor (Row-Major Order). Esto significa que el compilador reserva el espacio colocando la primera fila completa, inmediatamente después la segunda fila, y así sucesivamente en celdas consecutivas.

Un área de aplicación práctica idónea para este concepto es el modelado de mapas o tableros de juego en dos dimensiones. En este ejercicio, implementamos una versión simplificada del juego clásico de "Batalla Naval" (Battleship). Para lograrlo, utilizamos una matriz entera donde cada coordenada representa un estado dinámico en la lógica de juego. Mediante el uso de estructuras iterativas anidadas (ciclos for dentro de ciclos for), interactuamos con cada componente del mapa para inicializar variables, ocultar información estratégica y actualizar las respuestas lógicas según los eventos de entrada del usuario.

OBJETIVO

Objetivo General

Construir, estructurar e implementar un programa funcional en lenguaje C que haga uso de arreglos bidimensionales (matrices) para simular la mecánica principal de un juego de Batalla Naval, manipulando mapas numéricos mediante iteraciones anidadas y lógica de estados dinámicos.

Objetivos Particulares

- Implementar ciclos anidados `for` de dos niveles para realizar el recorrido, inicialización y renderizado gráfico abstracto de un tablero bidimensional de 5x5.

- Asignar y codificar estados numéricos en coordenadas específicas de la matriz para representar elementos ocultos (barcos) y registrar respuestas físicas de juego (agua e impactos).
- Diseñar una estructura cíclica de turnos interactivos que capture coordenadas ingresadas por el usuario, evalúe condiciones de juego en tiempo real y actualice el mapa impreso en la interfaz de la consola.

DESARROLLO

Para construir esta actividad de laboratorio, implementé una arquitectura algorítmica secuencial en el archivo `main.c`, dividiendo las tareas en cuatro etapas operacionales claras:

1. Inicialización y Codificación del Tablero

Declaré una matriz de tipo entero de dimensiones 5x5 llamada `tablero` y dos variables para guardar las coordenadas de disparo (`fila` y `columna`). El primer paso del flujo consiste en "limpiar" la memoria de la matriz; para ello programé dos ciclos `for` anidados que recorren cada combinación de filas (i) y columnas (j) asignándoles de forma obligatoria el valor de `0`, que en mi mapa de estados equivale a "Agua inexplorada".

2. Posicionamiento Oculto de Objetivos

Una vez limpio el tablero, procedí a simular la ubicación de la flota en el mapa. De forma manual y estática, modifiqué el contenido de tres coordenadas específicas asignándoles el valor de `1`, el cual representa de forma lógica la presencia de un "barco oculto":

- `tablero[1][2] = 1;`
- `tablero[3][4] = 1;`
- `tablero[0][0] = 1;`

Para la interfaz inicial del jugador, utilicé un doble ciclo que imprime únicamente símbolos `~`, ocultando visualmente la ubicación real de los barcos para preservar el misterio del juego.

3. Bucle Principal de Turnos y Evaluación de Disparos

Para la mecánica de interacción, envolví el núcleo del programa en un ciclo `for` parametrizado a 5 iteraciones exactas (`turno < 5`). En cada vuelta, se le solicita al usuario teclear una fila y una columna. Al recibir el dato en variables mediante `scanf`, el algoritmo ejecuta una validación selectiva basada en estados numéricos sobre la coordenada indicada (`tablero[fila][columna]`):

- **Si el valor es 1 (Impacto exitoso):** Se le avisa al jugador con la palabra `"impacto"` y la celda se sobrescribe con el número `3`, cambiando permanentemente el estado de esa casilla a "Barco hundido".
- **Si el valor es diferente a 1 (Fallo):** Se imprime `"agua"` en pantalla y la celda se actualiza con el número `2`, marcando el estado de la casilla como "Zona explorada sin éxito".

4. Renderizado Dinámico de la Interfaz

Al finalizar la evaluación de cada turno, el programa ejecuta un algoritmo de renderizado de mapa. Recorre la matriz completa y evalúa cada elemento mediante condicionales `if-else`: si el estado es `0` o `1` (agua o barco oculto), imprime un símbolo de mar (`~`); si el estado es `2`, imprime una marca de

fallo ('o '); y si es `3`, imprime una cruz de daño ('X '). Con esto se logra actualizar la pantalla en tiempo real mostrando al jugador el histórico de sus disparos.

El código fuente final de la aplicación se detalla a continuación:

```

#include <stdio.h>

int main() {
    int tablero[5][5];
    int fila, columna;

    // Inicializar tablero con ceros (Agua limpia)
    for(int i = 0; i < 5; i++) {
        for(int j = 0; j < 5; j++) {
            tablero[i][j] = 0;
        }
    }

    // Colocar barcos manualmente (Estado 1 = Barco)
    tablero[1][2] = 1;
    tablero[3][4] = 1;
    tablero[0][0] = 1;

    // Mostrar tablero inicial abstracto
    for(int i = 0; i < 5; i++) {
        for(int j = 0; j < 5; j++) {
            printf("~ ");
        }
        printf("\n");
    }

    // Ciclo principal del juego: 5 disparos permitidos
    for(int turno = 0; turno < 5; turno++) {
        printf("Fila: ");
        scanf("%d", &fila);

        printf("Columna: ");
        scanf("%d", &columna);

        // Evaluar el estado de la coordenada seleccionada
        if (tablero[fila][columna] == 1){
            printf("impacto\n");
            tablero[fila][columna] = 3; // Estado 3 = Impacto verificado
        } else {
            printf("agua\n");
            tablero[fila][columna] = 2; // Estado 2 = Disparo en agua
        }

        // Renderizado interactivo del tablero modificado
        for(int i = 0; i < 5; i++) {
            for(int j = 0; j < 5; j++) {
                if(tablero[i][j] == 0 || tablero[i][j] == 1) {
                    printf("~ "); // Mantiene el mar oculto
                } else if(tablero[i][j] == 2) {
                    printf("o "); // Imprime fallo explorado
                } else if(tablero[i][j] == 3) {
                    printf("X "); // Imprime acierto visual
                }
            }
            printf("\n");
        }
    }
}

```

```
    return 0;  
}
```

CONCLUSIÓN

El desarrollo de este simulador de juego me resultó de enorme utilidad para dominar de forma definitiva los arreglos de dos dimensiones en C. Al principio, entender las matrices puede parecer complicado por el uso de los dos índices, pero este ejercicio me dejó claro que trabajar con un mapa bidimensional no es más que usar un índice para controlar el eje vertical (las filas) y otro para el eje horizontal (las columnas).

Lo que me pareció más valioso e interesante de la práctica fue la implementación de la lógica de estados numéricos. Aprender a mapear conceptos abstractos como "agua", "barco oculto", "disparo fallido" o "impacto" usando simples números enteros (0, 1, 2 y 3) facilita enormemente el control de la aplicación mediante estructuras `if-else`. Además, me di cuenta de la importancia de usar adecuadamente los ciclos anidados para refrescar la interfaz en cada turno. Este proyecto me da una gran confianza para abordar problemas que requieran manipulación de imágenes, bases de datos tabulares u operaciones complejas con matrices matemáticas en los siguientes módulos del curso.

REFERENCIAS

- Kernighan, B. W., & Ritchie, D. M. (1988). *The C Programming Language* (2nd ed.). Prentice Hall.
- Deitel, P. J., & Deitel, H. M. (2014). *C How to Program* (7th ed.). Pearson.